



PHP ARRAYS 2025

Silvio Priem Mendes, PhD

PHP Structures

- Arrays
- Associative arrays
- Dates and Time
- Checking data types

PHP Arrays

- Arrays can have any size and contain any type of value.

```
$my_array[0] = 25;  
$my_array[1] = 'Bisons';
```

- Initialize an array:

```
$colors = array('red', 'green', 'blue');  
echo 'The 2nd color is ' . $colors[1];
```

PHP Arrays

- Full example

```
<html>
  <body>

    <?php
      /* First method to create array. */
      $numbers = array( 1, 2, 3, 4, 5);

      foreach( $numbers as $value ) {
        echo "Value is $value <br />";
      }

      /* Second method to create array. */
      $numbers[0] = 'one';
      $numbers[1] = 'two';
      $numbers[2] = 'three';
      $numbers[3] = 'four';
      $numbers[4] = 'five';

      foreach( $numbers as $value ) {
        echo "Value is $value <br />";
      }
    ?>

  </body>
</html>
```

PHP Associative Arrays

- The associative arrays are very similar to numeric arrays in term of functionality but they are different in terms of their index.
- Associative array will have their index as string so that you can establish a strong association between key and values.
- **NOTE:** Don't keep associative array inside double quote while printing otherwise it would not return any value.

PHP Associative Arrays

- PHP arrays are associative arrays which allow element values to be stored in relation to a key value rather than a strict linear index order.

```
$capitals['CO'] = 'Denver';  
$capitals['AR'] = 'Little Rock';
```

- Initialize an associative array:

```
$capitals = array('CO' => 'Denver', 'AR' => 'Little Rock');  
echo $capitals[CO]; // prints Denver
```

PHP Associative Arrays

```
<html>
  <body>

    <?php
      /* First method to associate create array. */
      $salaries = array('mohammad' => 2000, 'qadir' => 1000, 'zara' => 500);

      echo 'Salary of mohammad is '. $salaries['mohammad'] . '<br />';
      echo 'Salary of qadir is '. $salaries['qadir'] . '<br />';
      echo 'Salary of zara is '. $salaries['zara'] . '<br />';

      /* Second method to create array. */
      $salaries['mohammad'] = 'high';
      $salaries['qadir'] = 'medium';
      $salaries['zara'] = 'low';

      echo 'Salary of mohammad is '. $salaries['mohammad'] . '<br />';
      echo 'Salary of qadir is '. $salaries['qadir'] . '<br />';
      echo 'Salary of zara is '. $salaries['zara'] . '<br />';
    ?>

  </body>
</html>
```

PHP Associative Arrays

- Loop through an associative array:

```
<?php
```

```
$age = array('Peter'=>'35', 'Ben'=>'37', 'Joe'=>'43');
```

```
foreach($age as $x => $x_value) {  
    echo 'Key=' . $x . ', Value=' . $x_value;  
    echo '<br>';  
}
```

```
?>
```


PHP Multidimensional Arrays

- A multidimensional array is an array containing one or more arrays.
- PHP understands multidimensional arrays that are two, three, four, five, or more levels deep.
 - However, arrays more than three levels deep are hard to manage for most people.
- The dimension of an array indicates the number of indices you need to select an element:
 - For a two-dimensional array you need two indices to select an element
 - For a three-dimensional array you need three indices to select an element
 - Etc...

PHP Two-dimensional Arrays

- First, take a look at the following table:

Name	Stock	Sold
Volvo	22	18
BMW	15	13
Saab	5	2
Land Rover	17	15

- We can store the data from the table above in a two-dimensional array, like this:

```
$cars = array  
(  
    array('Volvo',22,18) ,  
    array('BMW',15,13) ,  
    array('Saab',5,2) ,  
    array('Land Rover',17,15) ,  
    array('Land Rover',17,15)  
);
```

PHP Two-dimensional Arrays

- Now the two-dimensional \$cars array contains four arrays, and it has two indices: row and column.
- To get access to the elements of the \$cars array we must point to the two indices (row and column):

```
echo $cars[0][0].': In stock: '.$cars[0][1].', sold: '.$cars[0][2].'.<br>;  
echo $cars[1][0].': In stock: '.$cars[1][1].', sold: '.$cars[1][2].'.<br>;  
echo $cars[2][0].': In stock: '.$cars[2][1].', sold: '.$cars[2][2].'.<br>;  
echo $cars[3][0].': In stock: '.$cars[3][1].', sold: '.$cars[3][2].'.<br>;
```

PHP Two-dimensional Arrays

- Looping the 2 dimensional array into list elements:

<?php

```
for ($row = 0; $row < 4; $row++) {  
    echo '<p><b>Row number $row</b></p>';  
    echo '<ul>';  
    for ($col = 0; $col < 3; $col++) {  
        echo '<li>'.$cars[$row][$col]. '</li>';  
    }  
    echo '</ul>';  
}
```

?>

PHP Common Array functions

- PHP sizeof() Function:

```
<?php
    $cars=array('Volvo','BMW','Toyota');
    echo sizeof($cars);
?>
```

returns the number of elements in an array

PHP Common Array functions

- PHP in_array() Function:

```
<?php
```

```
$people = array('Peter', 'Joe', 'Glenn', 'Cleveland');
```

```
if (in_array('Glenn', $people))
```

```
{
```

```
    echo 'Match found';
```

```
}
```

```
else
```

```
{
```

```
    echo 'Match not found';
```

```
}
```

```
?>
```

- The in_array() function searches an array for a specific value.
 - Note: If the search parameter is a string and the type parameter is set to TRUE, the search is case-sensitive.

PHP Common Array functions

- PHP array_search() Function:

```
<?php
```

```
$a=array('a'=>'red','b'=>'green','c'=>'blue');  
echo array_search('red',$a);
```

```
?>
```

- The array_search() function search an array for a value and returns the key.

PHP Date and Time

- The PHP Date() Function
- The PHP date() function formats a timestamp to a more readable date and time.
- Syntax:

date(format,timestamp)

Parameter	Description
format	Required. Specifies the format of the timestamp
timestamp	Optional. Specifies a timestamp. Default is the current date and time

PHP Date and Time

- Get a Simple Date

- The example below formats today's date in three different ways:

```
<?php
```

```
    echo 'Today is ' . date('Y/m/d') . '<br>' ;  
    echo 'Today is ' . date('Y.m.d') . '<br>' ;  
    echo 'Today is ' . date('Y-m-d') . '<br>' ;  
    echo 'Today is ' . date('l') ;
```

```
?>
```

PHP Date and Time

- **Get a Simple Date**

- The required format parameter of the date() function specifies how to format the date (or time).

- Characters that are commonly used for dates:

- d - Represents the day of the month (01 to 31)
- m - Represents a month (01 to 12)
- Y - Represents a year (in four digits)
- l (lowercase 'l') - Represents the day of the week
- Other characters, like '/', '.', or '-' can also be inserted between the characters to add additional formatting.

PHP Date and Time

- **PHP Tip - Automatic Copyright Year**
- Use the date() function to automatically update the copyright year on your website:

```
&copy; 2010-<?php echo date('Y');?>
```

PHP Date and Time

- **Get a Simple Time**
- Characters that are commonly used for times:
 - h - 12-hour format of an hour with leading zeros (01 to 12)
 - i - Minutes with leading zeros (00 to 59)
 - s - Seconds with leading zeros (00 to 59)
 - a - Lowercase Ante meridiem and Post meridiem (am or pm)
- The example below outputs the current time in the specified format:

```
<?php
echo 'The time is ' . date('h:i:sa');
?>
```

Note: the PHP date() function will return the current date/time of the server!

PHP Carbon Package

<https://carbon.nesbot.com>

```
<?php
    $dt = Carbon::now();
    $dt2 = Carbon::createFromDate(1987, 4, 23);

    echo $dt->addDays(2);
    echo $dt2->addMonths(3);
?>
```

PHP Checking Data Types

- **Data type conversion and test functions:**

- string **gettype(mixed var)** - get var's data type;
- bool **settype(mixed var, string type)** - change var's data type;
- **is_array()** - Checks whether the variable is an array;
- **is_double(), is_float(), is_real()** (All the same function) - Checks whether the variable is a float;
- **is_long(), is_int(), is_integer()** (All the same function) - Checks whether the variable is an integer;
- **is_string()** - Checks whether the variable is a string;

PHP Checking Data Types 2

- **Data type conversion and test functions:**
 - **is_bool()** - Checks whether the variable is a boolean;
 - **is_object()** - Checks whether the variable is an object;
 - **is_null()** - Checks whether the variable is null;
 - **is_scalar()** - Checks whether the variable is a scalar, that is, an integer, boolean, string, or float;
 - **is_numeric()** - Checks whether the variable is any kind of number or a numeric string;

PHP Checking Data Types 3

- **Test/change variable state:**
 - **bool `isset(mixed var)`** – returns true if the variable `var` is defined;
 - **void `unset(mixed var)`** – destroys the variable `var`;
 - **bool `empty(mixed var)`** – returns true if the variable `var` does not exist or is not initialized;



BIBLIOGRAPHY